

REST VS SOAP

Abbiamo semplicemente usato il metodo GET in tutte le richieste, fornendo tutti i parametri nella query string. Ciò ci ha consentito di provare facilmente i nostri ws tramite un browser, e richiamarli facilmente da codice Java. Nella realtà, è preferibile usare approcci più sofisticati. Nel panorama attuale si trovano principalmente due tipologie di web service:

- Web Service REST (approccio sempre più frequente)
- Web Service SOAP/WSDL (supportati dal W3C, in declino...)

REST = Representational State Transfer (trasferimento di stato rappresentazionale)

REST è uno stile architetturale, cioè un insieme di principi e vincoli da usare nella progettazione di un servizio. REST non è quindi un linguaggio, ma solo un insieme di "regole di progettazione".

Le web API e i ws che rispettano i principi REST sono dette RESTFull.

Il termine REST compare per la prima volta nel 2000, nella tesi di dottorato di Roy Thomas Fielding, uno dei creatori del protocollo HTTP.

Il server possiede un insieme di risorse (esempio: un evento sul calendario, un contatto, un articolo, un ordine)

il client può richiedere operazioni su queste risorse inviando al server una richiesta che contiene l'URL della risorsa

esempio: GET https://mia_web_api/eventi/16253

Il server risponde inviando al client una rappresentazione della risorsa (non è la risorsa stessa!). Ad esempio, una rappresentazione della risorsa può essere un file json oppure xml.

Usare i metodi HTTP (GET, POST, PUT, DELETE) , in modo adeguato, per differenziare le operazioni da effettuare su una risorsa

Le richieste GET devono essere SAFE (cioè non devono mai modificare lo stato delle risorse sul server)

Le richieste DELETE devono sempre essere IDEMPOTENTI (il risultato di due o più DELETE deve essere identico al risultato di una sola invocazione)

Usare in modo appropriato i codici di risposta HTTP anzichè inventare nuovi codici e realizzare operazioni STATELESS: ogni richiesta deve contenere tutte le informazioni che servono al server per gestire la risposta stessa (il server non deve dover "ricordare" le richieste passate)

Create --> richiesta HTTP POST

Read --> richiesta HTTP GET (deve essere safe)

Update --> richiesta HTTP UPDATE

Delete --> richiesta HTTP DELETE (deve essere idempotente)

SOAP (Simple Object Access Protocol) è un protocollo di comunicazione standardizzato che permette agli applicativi di scambiarsi informazioni strutturate tramite servizi web. Le specifiche di SOAP sono mantenute da W3C (World Wide Web Consortium).

Caratteristiche principali di SOAP:

- **Formato del Messaggio:** SOAP utilizza XML per formattare i dati dei messaggi, garantendo così una trasmissione di dati precisa e rigorosamente tipizzata.

- **Indipendenza dal Trasporto:** Sebbene SOAP sia comunemente utilizzato su HTTP, supporta diversi altri protocolli di trasporto come SMTP, TCP, e altri.
- **Sicurezza:** SOAP supporta standard di sicurezza come crittografia, autenticazione e integrità del messaggio.
- **Transazionalità:** Le estensioni di SOAP supportano la gestione delle transazioni, assicurando che le operazioni possano essere completate con successo o completamente annullate in caso di errore

WSDL definisce un formato standard per descrivere in modo formale le operazioni, i parametri e i formati di risposta di un servizio web. E' comunemente usato in combinazione con SOAP per fornire una descrizione dettagliata delle operazioni supportate dai servizi web SOAP.

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```

▼ <wsdl:definitions xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" xmlns:tm="http://microsoft.com/wsdl/mime
  xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/" xmlns:mime="http://schemas.xmlsoap.org/wsdl/mime/"
  xmlns:s="http://www.w3.org/2001/XMLSchema" xmlns:soap12="http://schemas.xmlsoap.org/wsdl/soap12/" xmlns:http=
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/" targetNamespace="http://tempuri.org/">
  ▼ <wsdl:types>
    ▼ <s:schema elementFormDefault="qualified" targetNamespace="http://tempuri.org/">
      ▼ <s:element name="Add">
        ▼ <s:complexType>
          ▼ <s:sequence>
            <s:element minOccurs="1" maxOccurs="1" name="intA" type="s:int"/>
            <s:element minOccurs="1" maxOccurs="1" name="intB" type="s:int"/>
          </s:sequence>
        </s:complexType>
      </s:element>
      ▼ <s:element name="AddResponse">
        ▼ <s:complexType>
          ▼ <s:sequence>
            <s:element minOccurs="1" maxOccurs="1" name="AddResult" type="s:int"/>
          </s:sequence>
        </s:complexType>
      </s:element>
      ▼ <s:element name="Subtract">
        ▼ <s:complexType>
          ▼ <s:sequence>
            <s:element minOccurs="1" maxOccurs="1" name="intA" type="s:int"/>
            <s:element minOccurs="1" maxOccurs="1" name="intB" type="s:int"/>
          </s:sequence>
        </s:complexType>
      </s:element>
      ▼ <s:element name="SubtractResponse">
        ▼ <s:complexType>
          ▼ <s:sequence>
            <s:element minOccurs="1" maxOccurs="1" name="SubtractResult" type="s:int"/>
          </s:sequence>
        </s:complexType>
      </s:element>
    </s:schema>
  </wsdl:types>

```

REST e SOAP sono due diversi paradigmi utilizzati nella progettazione di servizi web.

Facciamo ora una differenza tra i due secondo diverse caratteristiche:

Architettura:

- **REST** è uno stile architettonico che utilizza HTTP. Non è legato a nessun protocollo, ma segue principi come l'uso di URI per identificare risorse e metodi HTTP (GET, POST, PUT, DELETE) per operarci sopra.
- **SOAP** è un protocollo più formale che definisce un messaggio rigoroso e regole di scambio, usando XML per il formato del messaggio. Può operare su diversi protocolli come HTTP, SMTP, TCP, e altri.

Flessibilità:

- **REST** è generalmente considerato più semplice e flessibile. Gli sviluppatori possono facilmente lavorare con REST usando gli strumenti e le librerie disponibili per HTTP.
- **SOAP** richiede più disciplina e standardizzazione, il che può essere vantaggioso in ambienti aziendali dove i contratti di interfaccia formali sono necessari.

Sicurezza:

- **REST** può usare sicurezza a livello di trasporto come HTTPS, ma non ha standard di sicurezza integrati al livello di SOAP.
- **SOAP** supporta WS-Security, che include specifiche per la sicurezza più robuste come integrità del messaggio e privacy, oltre a estensioni per transazioni, affidabilità dei messaggi e altro.

Applicazioni:

- **REST** è ampiamente usato per applicazioni web pubbliche, API mobili, e servizi che richiedono integrazione agile e scalabile.
- **SOAP** è spesso preferito in ambienti aziendali che richiedono livelli elevati di formalità nelle transazioni e sicurezza, come nei servizi finanziari o nelle telecomunicazioni.

Flessibilità:

- **REST** è generalmente considerato più semplice e flessibile. Gli sviluppatori possono facilmente lavorare con REST usando gli strumenti e le librerie disponibili per HTTP.
- **SOAP** richiede più disciplina e standardizzazione, il che può essere vantaggioso in ambienti aziendali dove i contratti di interfaccia formali sono necessari.

Esempi di Web Services che utilizzano REST

API di Google Maps:

Twitter API:

Amazon Simple Storage Service (S3)

Spotify Web API

GitHub API:

Esempi di Web Services che utilizzano SOAP

API di servizi finanziari

API di servizi sanitari

SSO (Single Sign-On) Services: